



PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO
FACULTAD DE INGENIERÍA
ESCUELA DE INGENIERÍA INFORMÁTICA

Sistema de Gestión y Asignación de Turnos de Enfermería

Integrantes:

Rodrigo Escudero

Ariel Fuenzalida

Matias Santalices

11 de septiembre de 2026

Índice

Lista de figuras	3
Lista de tablas	4
Introducción	5
1 ¿Qué datos maneja el sistema? (SIA-1)	6
2 Las clases que construyen el sistema (SIA-2)	7
2.1 Clase Persona (datos básicos de cualquiera)	7
2.2 Clase Enfermera (una profesional concreta)	7
2.3 Jerarquía de Turnos (diferentes asignaciones laborales)	7
2.4 Clase AreaHospitalaria (el conjunto de personal)	7
3 Buenas prácticas (SIA-3)	9
4 Uso de colecciones de Java (SIA-4)	10
4.1 1. Primer nivel - El listado global de enfermeras	10
4.2 2. Segundo nivel - La lista de turnos dentro de cada enfermera	10
5 Sobrecarga de métodos (SIA-5)	11
6 Sobreescritura y polimorfismo (SIA-6)	12
7 Funcionalidades básicas del menú (SIA-7)	13
7.1 Modo Consola	13
7.2 Modo Gráfico	13
8 Editar, eliminar y buscar (SIA-8)	15
8.1 Búsqueda	15
8.2 Edición	15
8.3 Eliminación	15
9 Funcionalidad propia del programa (SIA-9)	16
9.1 1. Filtro de sobrecarga laboral	16
9.2 2. Dashboard visual de especialidades	16
10 Consola o ventanas, a elección (SIA-10)	17
11 Los datos no se pierden al cerrar (SIA-11)	18
12 Manejando errores sin que el programa se caiga (SIA-12)	19
13 Requisitos Opcionales (SIA-O4 y SIA-O1)	20
14 UML completo del programa	21

15 Control de versiones con GitHub (SIA-13)	22
--	-----------

Índice de figuras

Figura 2.1	Diagrama de Clases del sistema de Turnos	8
Figura 4.1	Estructura de colecciones en memoria del sistema de turnos de enfermería	10
Figura 7.1	Vista previa del menú en modo ventana	14
Figura 9.1	Visualización del reporte gráfico	16
Figura 10.1	Captura del Selector de Interfaz Gráfica vs Consola	17
Figura 14.1	Diagrama UML completo del programa	21
Figura 15.1	Número de commits del repositorio en GitHub	22

Índice de tablas

Tabla 12.1	Excepciones personalizadas del sistema	19
------------	--	----

Introducción

El presente informe detalla el diseño, desarrollo e implementación del **Sistema de Gestión y Asignación de Turnos de Enfermería**, una aplicación construida utilizando el lenguaje de programación Java bajo el paradigma de Orientación a Objetos.

En el entorno hospitalario, la correcta asignación de turnos, el manejo de licencias y el seguimiento de horas trabajadas por el personal de enfermería son tareas críticas. Este sistema busca solucionar la gestión manual de esta información, proveyendo un software capaz de almacenar, filtrar y persistir datos eficientemente a través del patrón de arquitectura Modelo-Vista-Controlador (MVC).

A lo largo del documento, se exploran las decisiones técnicas adoptadas para cumplir con los requerimientos del Sistema de Información Académica (SIA) de la asignatura Programación Avanzada, destacando la implementación de Colecciones, Polimorfismo, Manejo de Excepciones y Persistencia.

1. ¿Qué datos maneja el sistema? (SIA-1)

Para administrar correctamente un hospital, necesitamos representar tres cosas principales:

- **Enfermera:** tiene un RUT (que la identifica de forma única), nombre completo, años de experiencia y la especialidad médica a la que se dedica.
- **Turno:** es un bloque de tiempo de trabajo asignado. Guarda la fecha, la hora de inicio, la hora de fin y las horas totales que se trabajaron. Según el tipo de turno, guarda información extra (por ejemplo, a qué cama fue asignada o el motivo de una licencia).
- **Área Hospitalaria:** agrupa a todas las enfermeras que trabajan en un mismo departamento (por ejemplo, "Urgencias").

La relación entre ellos es lógica y sencilla: un **Área Hospitalaria** contiene varias **Enfermeras**, y cada **Enfermera** guarda una lista con todos los **Turnos** que ha realizado a lo largo de su carrera.

2. Las clases que construyen el sistema (SIA-2)

A continuación se muestran las clases Java que modelan los datos del hospital. Cada una guarda información clave y provee métodos para realizar acciones cotidianas.

2.1. Clase Persona (datos básicos de cualquiera)

¿Qué hace? Guarda los datos que cualquier persona en el hospital tiene, independientemente de su profesión (RUT y Nombre).

2.2. Clase Enfermera (una profesional concreta)

¿Qué hace? Extiende a **Persona** añadiendo su experiencia, especialidad médica y, lo más importante, una lista donde se irán guardando todos sus turnos. Proporciona métodos para agregar un turno nuevo o mostrar su historial completo.

2.3. Jerarquía de Turnos (diferentes asignaciones laborales)

¿Qué hace? Turno es una clase genérica (abstracta) que define lo que es común a toda jornada: fecha y horas trabajadas. Luego tenemos tres variantes:

- **TurnoRegular:** añade el número de cama a la que fue asignada la enfermera.
- **Licencia:** añade el motivo médico por el que no pudo asistir a trabajar.
- **CambioTurno:** añade el nombre de la enfermera de reemplazo.

Cada una de estas variantes sabe dar un resumen de sí misma cuando se le pide, devolviendo un texto adecuado a su situación.

2.4. Clase AreaHospitalaria (el conjunto de personal)

¿Qué hace? Representa el departamento del hospital. Internamente utiliza una estructura de datos (**TreeMap**) que asocia el RUT con el objeto **Enfermera**, manteniéndolas ordenadas automáticamente por su identificación para encontrarlas rápidamente.

UML de Clases - TurnosEnfermeria

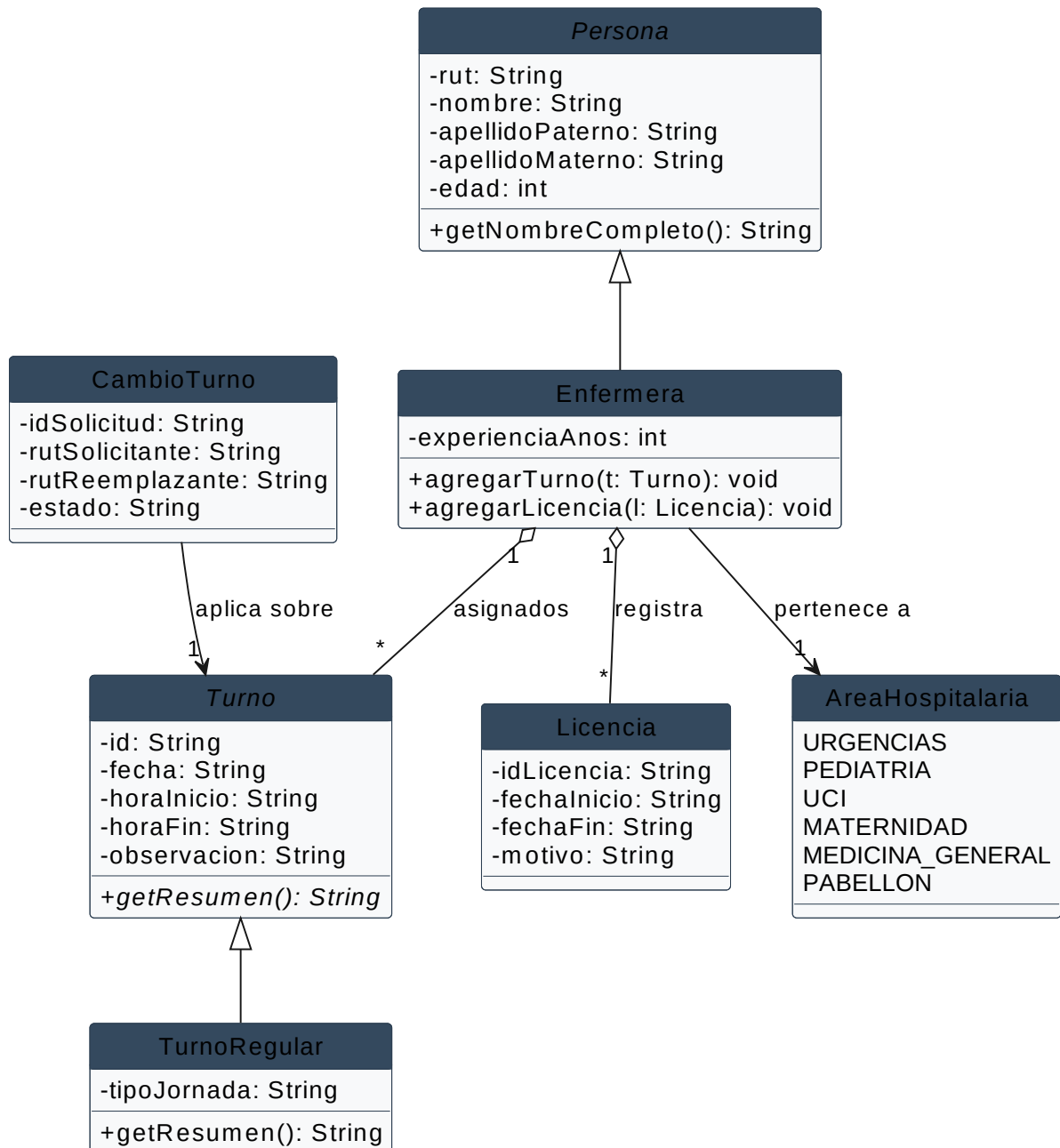


Figura 2.1: Diagrama de Clases del sistema de Turnos

3. Buenas prácticas (SIA-3)

Para que el programa sea fácil de leer y de mantener en el futuro, nos aseguramos de seguir ciertas reglas clave:

- **Atributos privados:** todas las clases protegen su información utilizando **private**, permitiendo que solo se modifique a través de los **getters** y **setters** autorizados.
- **Código modular:** separamos claramente el programa en partes lógicas. Lo visual (menús) no se mezcla con el funcionamiento interno (clases) ni con el guardado de archivos.
- **Datos iniciales:** para no tener que empezar con el hospital vacío cada vez que abrimos el programa, el sistema lee automáticamente la información desde un archivo base cuando arranca.

4. Uso de colecciones de Java (SIA-4)

El sistema guarda la información en dos niveles de colecciones, organizando la jerarquía desde el departamento hospitalario hasta el historial de cada empleada:

4.1. 1. Primer nivel - El listado global de enfermeras

Se utiliza un **TreeMap** donde la **clave** es el RUT y el **valor** es toda la información de la **Enfermera**. Al usar **TreeMap**, las enfermeras quedan ordenadas por RUT automáticamente y podemos encontrar a cualquiera en el hospital rápidamente sin tener que revisar la lista una por una.

4.2. 2. Segundo nivel - La lista de turnos dentro de cada enfermera

Cada **Enfermera** tiene, en su interior, un **ArrayList** que guarda sus registros de turnos. Usamos una lista simple porque queremos que los turnos queden guardados exactamente en el orden cronológico en el que se fueron agregando a su historial.

Anidamiento: El Área Hospitalaria contiene a las enfermeras, y cada enfermera contiene su lista de turnos. Es, literalmente, una colección de datos viviendo dentro de otra colección.

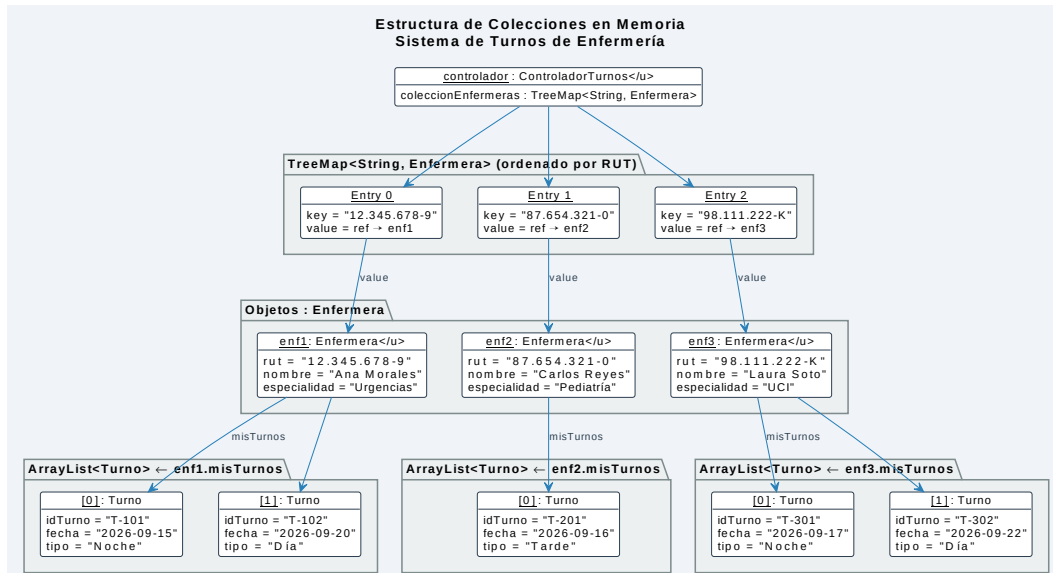


Figura 4.1: Estructura de colecciones en memoria del sistema de turnos de enfermería

5. Sobrecarga de métodos (SIA-5)

La sobrecarga consiste en poder darle la misma instrucción al sistema de distintas maneras, dependiendo de qué tan específicos queramos ser. El sistema decide qué hacer según lo que le pasemos.

En nuestro controlador de `AreaHospitalaria`, lo vemos claramente cuando pedimos la lista de enfermeras:

- Si le decimos simplemente al sistema "**listarEnfermeras()**" sin darle más detalles, entenderá que queremos ver a **todas** las profesionales ingresadas hasta el momento.
- En cambio, si le decimos "**listarEnfermeras(120)**", el sistema nota que le pasamos un número. Entonces asume que queremos **filtrar**, mostrando solo a aquellas enfermeras que han trabajado más de 120 horas.

Esto hace que el sistema sea más intuitivo, evitando tener que memorizar nombres de instrucciones distintas para tareas que, en el fondo, son casi iguales.

6. Sobreescritura y polimorfismo (SIA-6)

Cuando una clase hija redefine cómo hacer algo que su clase padre le enseñó, hablamos de **sobreescritura**. En nuestro sistema de turnos, la clase base **Turno** indica que toda jornada debe poder dar un resumen de sí misma. Sin embargo, cada subclase lo cambia para dar su propio mensaje:

- **TurnoRegular:** responde con "Turno regular asignado a la Cama N°X".
- **Licencia:** responde con "Licencia médica tomada por el motivo: X".
- **CambioTurno:** responde con "Turno modificado. Reemplaza: Enfermera X".

Polimorfismo en acción: La gran ventaja lógica de esto es que, cuando queremos imprimir el historial completo de una enfermera, no tenemos que revisar si el turno del día martes fue una Licencia o un Turno Regular para imprimir el mensaje correcto.

Simplemente recorremos la lista de turnos, y con una sola línea de código, Java le pide el resumen al registro. El sistema ejecuta automáticamente la versión correcta según el tipo real del objeto. Así, el código es más limpio y funciona para cualquier tipo de turno que inventemos en el futuro.

7. Funcionalidades básicas del menú (SIA-7)

El sistema ofrece, tanto en consola como en interfaz gráfica, las operaciones necesarias para ingresar manualmente al personal y visualizar lo que hemos guardado.

7.1. Modo Consola

El menú principal de texto incluye opciones como:

- **Opción 1 - Agregar Enfermera:** el sistema solicita los datos de la profesional, verifica que el RUT sea correcto, y la guarda en la colección global del hospital.
- **Opción 2 - Listar Enfermeras:** recorre la colección y muestra en pantalla a todo el personal médico registrado.
- **Opción 3 - Asignar Turno:** pide el RUT de la enfermera a la que queremos asignarle el horario, pregunta las fechas y horas, y agrega el registro al historial de esa persona.
- **Opción 5 - Ver Historial de Turnos:** muestra cronológicamente todos los turnos y licencias que ha tenido una enfermera, usando el método polimórfico para dar el resumen correcto de cada uno.

7.2. Modo Gráfico

- **Panel Ingreso:** un formulario amigable con cajas de texto para agregar una nueva enfermera sin usar comandos.
- **Panel Gestión de Personal:** contiene una tabla donde se listan todas las enfermeras cargadas desde el sistema, permitiendo ver de un vistazo el personal disponible.
- **Panel Asignaciones:** al cargar el historial de una enfermera, se llena una tabla con todos sus registros de asistencia y turnos, ofreciendo un botón directo para ingresar uno nuevo.

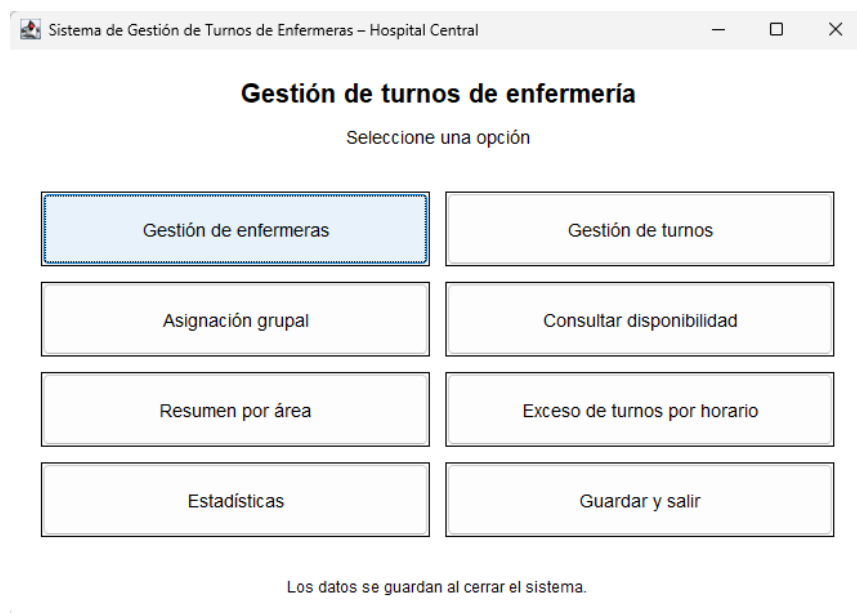


Figura 7.1: Vista previa del menú en modo ventana

8. Editar, eliminar y buscar (SIA-8)

Además de agregar y listar, el sistema nos permite actuar sobre la información que ya existe, manteniendo los datos siempre actualizados:

8.1. Búsqueda

- **Enfermera:** podemos buscar a un miembro del personal por su RUT y el sistema nos traerá toda su ficha de inmediato (gracias a la eficiencia de la estructura **TreeMap**).

8.2. Edición

- **Enfermera:** podemos modificar los años de experiencia o cambiar la especialidad de una profesional. Esto es vital si una enfermera se certifica en una nueva área sin perder su historial antiguo de turnos.

8.3. Eliminación

- **Enfermera:** podemos dar de baja a una enfermera del sistema. Al borrar su RUT del registro global, también se eliminan automáticamente todos sus turnos asociados.
- **Turno específico:** podemos eliminar un turno mal ingresado del historial de una enfermera. Al hacerlo, el sistema también se encarga de restar esas horas del conteo total trabajado por la profesional, manteniendo los reportes cuadrados.

9. Funcionalidad propia del programa (SIA-9)

Además de las operaciones básicas de mantención de datos, el sistema incluye herramientas pensadas específicamente para facilitar la administración hospitalaria y cuidar al personal médico.

9.1. 1. Filtro de sobrecarga laboral

Esta herramienta ayuda a detectar tempranamente al personal que está trabajando horas excesivas, previniendo el síndrome de *burnout*.

¿Cómo funciona? El usuario ingresa un número límite de horas (por ejemplo, 120 horas mensuales). El sistema recorre a todas las enfermeras y revisa la sumatoria de las horas de sus turnos. Si el total supera el límite ingresado, el sistema alerta mostrando el nombre de la enfermera y su total de horas. Esto aprovecha los datos que ya están en el sistema sin necesidad de modificar la base de datos.

9.2. 2. Dashboard visual de especialidades

Para tener una visión administrativa clara del tipo de personal contratado, el sistema genera dinámicamente un gráfico de distribución (Gráfico de Torta).

El sistema clasifica a las enfermeras según su especialidad médica (UCI, Pediatría, Urgencias, etc.) y calcula los porcentajes. Esta información visual le permite al administrador del hospital saber si hay un déficit de personal en alguna área específica de forma instantánea.

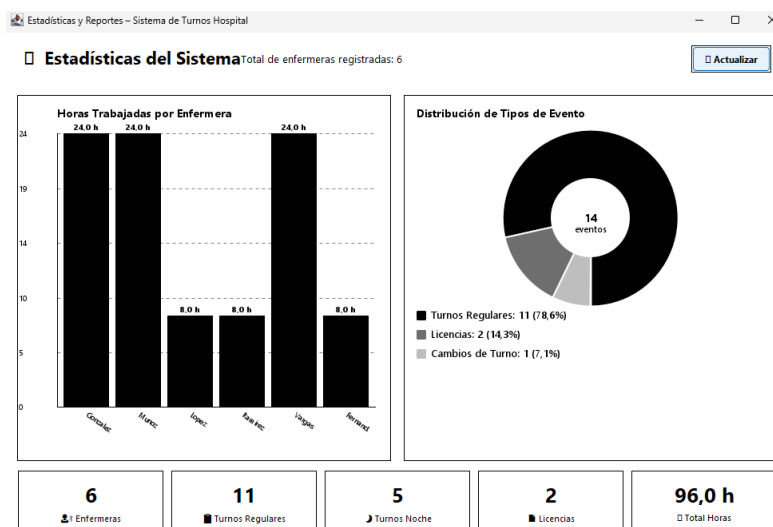


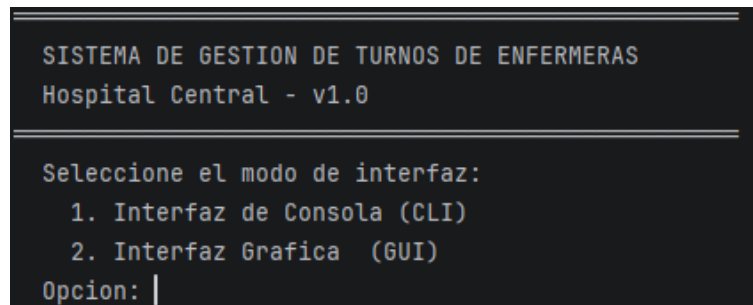
Figura 9.1: Visualización del reporte gráfico

10. Consola o ventanas, a elección (SIA-10)

El diseño de la aplicación respeta la separación de intereses del MVC. Gracias a esto, el mismo Controlador y Modelo pueden ser consumidos por diferentes Vistas sin alterar su lógica interna.

Al iniciar el programa (clase **Main**), el sistema pregunta al usuario el modo de arranque deseado.

- **Modo Consola:** Una interfaz rápida basada en texto, controlada mediante la entrada estándar **Scanner**.
- **Modo Interfaz Gráfica (GUI):** Una experiencia visual e interactiva construida con la librería **Swing**, diseñada con un tema oscuro estético y componentes personalizados como **JFrame**, **JPanel**, y **JTable**.



```
SISTEMA DE GESTION DE TURNOS DE ENFERMERAS
Hospital Central - v1.0

Seleccione el modo de interfaz:
  1. Interfaz de Consola (CLI)
  2. Interfaz Grafica  (GUI)
Opcion: |
```

Figura 10.1: Captura del Selector de Interfaz Gráfica vs Consola

11. Los datos no se pierden al cerrar (SIA-11)

Toda la información gestionada durante el tiempo de ejecución se asegura mediante persistencia en archivos de texto (archivos por lotes **CSV**).

La clase utilitaria **GestorArchivos** se encarga de leer los archivos **enfermeras.csv** y **turnos.csv** (ubicados en **src/main/resources/**) durante la inicialización del controlador. Asimismo, durante la rutina de salida del menú principal, el gestor escribe y sobrescribe estos archivos, previniendo la pérdida de los nuevos registros.

12. Manejando errores sin que el programa se caiga (SIA-12)

Para evitar que el programa se cierre abruptamente si un usuario ingresa mal un RUT o asigna un horario que no corresponde, el sistema utiliza el manejo preventivo de excepciones.

Hemos definido **dos excepciones personalizadas** para controlar reglas lógicas del negocio:

Excepción	¿Cuándo se lanza?
RutInvalidoException	Al ingresar un RUT que no cumple con el formato o cuyo dígito verificador chileno no calza.
TurnoConflictoException	Cuando intentamos asignarle a una enfermera un turno nuevo que choca directamente con el horario de un turno que ya tenía asignado.

Tabla 12.1: Excepciones personalizadas del sistema

Ejemplo del flujo de prevención (RUT inválido):

1. El usuario intenta ingresar un RUT mal escrito al registrar una enfermera nueva.
2. El sistema revisa el formato, se da cuenta del error y **lanza** la `RutInvalidoException`.
3. En lugar de colapsar la aplicación, el bloque de código **captura** el error en la interfaz (Consola o Ventana).
4. El sistema muestra un mensaje claro: ".^{EI} RUT ingresado no es válido".
5. El programa continúa su ejecución normal permitiendo al usuario intentarlo de nuevo.

13. Requisitos Opcionales (SIA-O4 y SIA-O1)

Además de los requisitos mínimos aprobatorios, el sistema incorpora módulos extra que complementan la experiencia del usuario y facilitan la lectura a futuros programadores:

- **Gráficos Estadísticos (SIA-O1):** Como se mencionó en la funcionalidad del negocio, la interfaz incluye paneles que dibujan gráficos estadísticos reales. Esto aporta un inmenso valor a la administración, permitiendo tomar decisiones visuales rápidas sobre la contratación de personal en base a la distribución de especialidades.
- **Documentación Clara (SIA-O4):** Todo el código fuente cuenta con etiquetas Javadoc explicativas. Esto significa que si otro programador intenta expandir este sistema hospitalario en el futuro, no tendrá que adivinar qué hace cada función; el mismo código le entregará un manual estructurado explicando los parámetros requeridos y el comportamiento esperado.

14. UML completo del programa

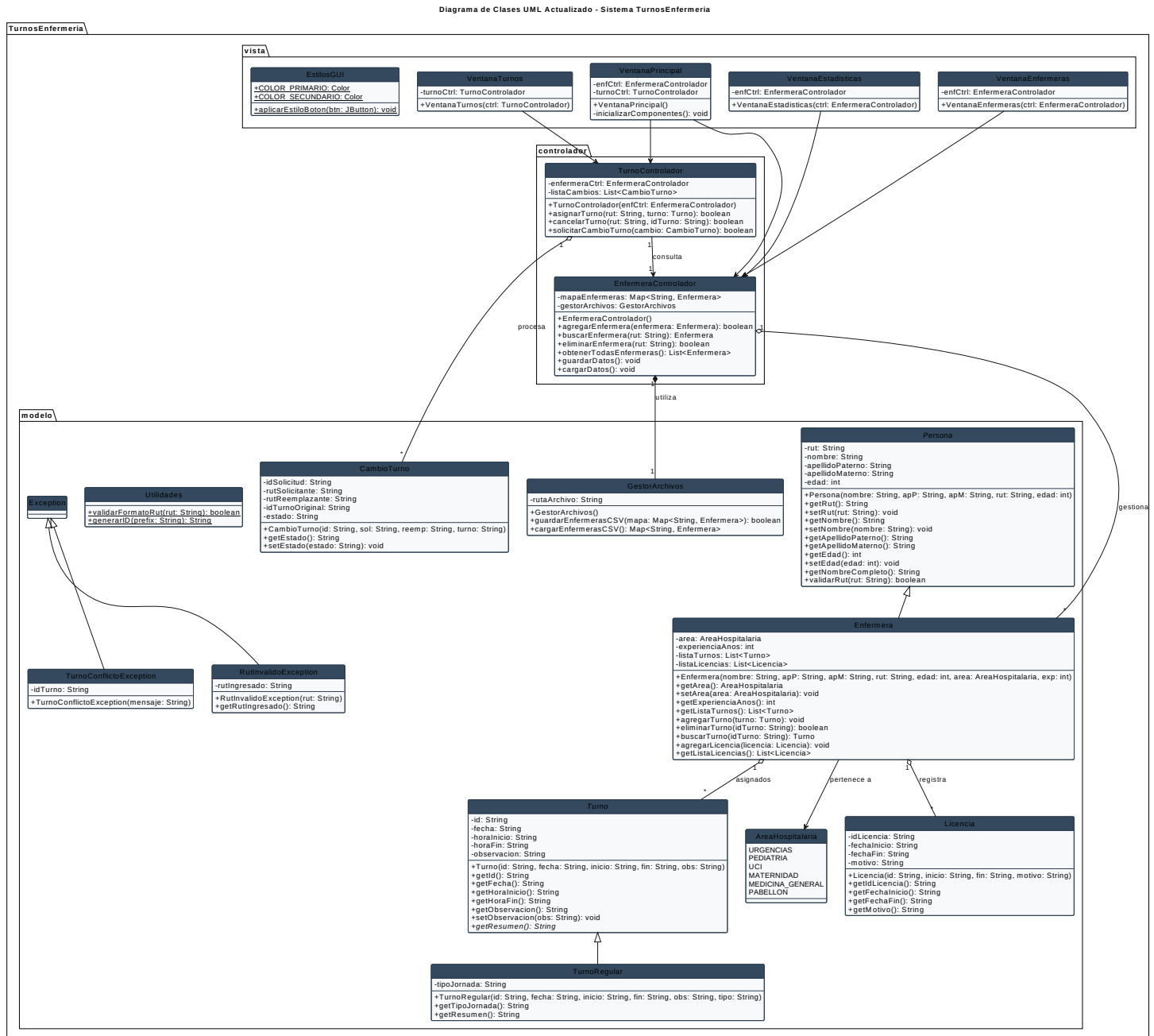


Figura 14.1: Diagrama UML completo del programa

15. Control de versiones con GitHub (SIA-13)

El proceso de desarrollo, iteración y control de código fuente fue llevado completamente a través de la herramienta **Git** en conjunción con la plataforma **GitHub**.

Se utilizó un sistema de ramas (Branches) para la implementación de nuevas características y fixeo de bugs (hotfixes). El repositorio del proyecto es de acceso público y el historial completo de **commits** se encuentra disponible en:

<https://github.com/YungRodri/Borrador-proyecto-enfermeras>

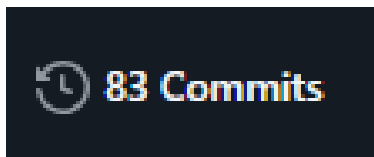


Figura 15.1: Número de commits del repositorio en GitHub